

Donation Campaigns

phpBB Extension — Complete Documentation

Version 1.0.0-beta1 · 2026-07-22

Contents

Donation Campaigns

- What it does
- What it does not do
- Requirements
- Installation
- Permissions
- Configuration
- Campaign workflow
- Donation workflow
- Known limitations
- Licence

Administrator guide

- Before you start
- 1. Install and enable
- 2. Grant the permissions
- 3. Configure the currency
- 3a. ⚠ Before you merge or split a topic
- 4. Create a campaign
- 5. Record a payment
- 6. Decide whether to name the donor
- 7. Edit or delete a confirmed entry
- 8. Recalculate a total
- 9. Disable or archive a campaign
- 9a. Where actions are logged
- 10. What happens when a topic or forum is deleted
- Troubleshooting

Privacy and data handling

- What is stored
- What is not stored
- The donor name is the sensitive part
- Deletion and erasure
- Logging

Donation Campaigns — Release Notes

- 1.0.0-beta1
- What the extension does
- Headline change — management moved from the ACP to the topic
- Localisation
- Major architectural decisions
- Known limitations
- Known issue — RELEASE-BLOCKING before 1.0 final
- Upgrade / migration

Why beta

Developer notes

Package

Layers

Money

The campaign total

Transaction boundaries

Confirmed-donation semantics

The action button

Deletion cascades

Shadow topics

phpBB integration points

The escaping contract

The BBCode description

Styles

Tests

Local integration environment

EPV and coding standards

Donation Campaigns

A phpBB 3.3 extension that attaches a fundraising campaign to a topic and shows its progress above the first post.

- [Administrator guide](#) — the day-to-day workflow
 - [Privacy and data handling](#) — what is stored, and what publishing a donor name means
 - [Developer notes](#) — architecture, contracts, tests
-

What it does

An administrator — or an authorised forum moderator — creates a campaign, points it at an existing topic, and sets a target amount. The topic then shows a box above its first post with the target, the amount collected, a progress bar, and — optionally — the number of donations and the names of donors who agreed to be named.

As money arrives, an administrator or an authorised forum moderator records each payment from the topic. The public total is the sum of those records and nothing else.

What it does not do

Read this part before installing. It is the difference between what this extension is and what people often assume a "donation extension" is.

- **It does not process payments.** No PayPal, no Stripe, no card handling, no webhooks, no callbacks. The extension never contacts a payment provider and makes no outbound requests of any kind.
- **It records confirmed receipts only.** A donation row means money that has *already arrived* and that an administrator has verified. It is bookkeeping, not a transaction.
- **No visitor or member can submit or confirm a donation.** There is no public form. Every entry is made from the topic by someone holding the appropriate permission — an administrator, or a forum moderator granted the forum-scoped donation permission for that forum.
- **It does not verify anything.** It cannot check a bank statement or a provider's records. Whether the money truly arrived is the administrator's judgement, and the extension records that judgement.
- **It stores no payment data** — no account numbers, no IBANs, no transaction identifiers, no card details, no proof-of-payment files. See [PRIVACY.md](#).
- **The action button is a plain external link.** Each campaign sets its own button text and destination — "How to donate", "Request bank details", "Über PayPal spenden". The destination may be a PayPal link, a page of bank instructions, another phpBB topic, a GoFundMe page or any other `https://` address. It is a link and nothing more: no provider script, no embedded form, no logo, no HTML. Whoever clicks it leaves the board, and the administrator still confirms the money by hand afterwards.

Using PayPal, or anything else

There is nothing to configure and no integration to enable. Create a hosted donation link in your own PayPal account, paste it into the campaign's **Donation link**, and set **Link text** to whatever the button should say — for example *Donate via PayPal*. The same field takes a GoFundMe or Betterplace page, a bank-information page, an internal phpBB topic, or any other valid `https://` address, and each campaign can point somewhere different.

The extension treats every one of these as an ordinary external link. It does not know which provider a URL belongs to, stores no merchant identifier, and performs no verification: whoever clicks the button leaves the board, and the

administrator records the money by hand afterwards. That is the whole design, and it is intentional rather than unfinished — see [DEVELOPERS.md](#).

Requirements

phpBB	3.3.16 or later , below 4.0. Enforced by <code>ext.php</code> ; installing on an older 3.3.x fails cleanly
PHP	8.2 or newer
Database	See below
Style	prosilver
Language	English and German

Verified versus designed-for

Stating this precisely matters more than making the table look full.

	Actually executed	Designed for, not executed
phpBB	3.3.17	3.3.16 (the supported floor, but never run)
PHP	8.2 (board), 8.2/8.3 (tests)	8.4 — permitted by <code>>=8.2</code> , not run
Database	SQLite 3, MariaDB 10.11	MySQL, PostgreSQL, MS SQL Server, Oracle

The extension uses phpBB's DBAL throughout and writes no database-specific SQL, so the untested engines are expected to work — but "expected" is not "verified", and this table will not claim otherwise.

prosilver is the only officially supported style (ADR-013). Styles that inherit from prosilver may work; that is neither tested nor supported.

Installation

1. Copy the package to `ext/uflagmey/donationcampaigns/` in your board.
2. **ACP** → **Customise** → **Extensions** → **Donation Campaigns** → **Enable**.

Or from the command line:

```
php bin/phpbbcli.php extension:enable uflagmey/donationcampaigns
php bin/phpbbcli.php cache:purge
```

Lifecycle

Action	Effect
Enable	Creates two tables, four configuration settings, the three permissions and their permission category, and the ACP menu
Disable	Hides the campaign box and the ACP menu. All data is kept , and re-enabling restores everything

Action	Effect
Purge	Destroys all campaigns and donations , and removes the tables, settings and permissions. This cannot be undone
Upgrade	Replace the files and run <code>php bin/phpbbcli.php db:migrate</code> . Migrations are idempotent and safe to re-run

Take a database backup before purging. "Disable" is the reversible one; "purge" is not.

Permissions

Three permissions, grouped under a dedicated **Donation Campaigns** category in the permissions UI:

- **a_donationcampaigns** — administrator, **global**. Full access: the ACP settings and read-only oversight lists, the admin-only maintenance (recalculate a total; hard-delete a non-empty campaign), and — as an override — every frontend campaign and donation action on every forum. Granted to `ROLE_ADMIN_FULL` and `ROLE_ADMIN_STANDARD` on installation, if those roles exist.
- **m_donationcampaigns_manage** — moderator, **forum-scoped**. Lets the holder manage the campaign *shell* on topics in that forum: create, edit, enable/disable, and delete an *empty* campaign. It does **not** grant donation management.
- **m_donationcampaigns_donations** — moderator, **forum-scoped**. Lets the holder manage the donation *ledger* on topics in that forum: add, edit and delete confirmed donations. ⚠ This permission exposes donor names, private donor identities and confirmed amounts — grant it only to people you trust with that personal data.

The two `m_` permissions are **independent**: holding one does not grant the other. `a_donationcampaigns` overrides both everywhere; it is not granted on install to non-admins.

Granting either `m_` permission makes the grantee a forum moderator. phpBB defines a forum moderator as anyone holding an `m_` permission on that forum, so a user or group you grant a donation permission to appears in that forum's **Moderator** list on topic and forum views and gains moderator standing there. This is inherent to phpBB and is not a silent, invisible grant — treat it as promoting the grantee to a limited moderator of that forum.

Configuration

ACP → Extensions → Donation campaigns → Settings

Setting	Default	Notes
Currency code	EUR	Three letters, e.g. EUR , USD , GBP
Currency symbol	€	Up to 10 characters
Decimal places	2	0–4. 0 for yen, 3 for dinar
Donors listed	25	1–500, before the box summarises the rest

⚠ **Changing "Decimal places" after donations exist changes how every stored amount is read, and converts nothing.** An amount stored as `1000` shows as `10.00` at two places and `1.000` at three. The extension refuses the change unless you confirm it. See the [administrator guide](#).

Campaign workflow

Campaigns are managed **from the topic**, not the ACP.

1. Open the topic the campaign is for.
2. **Topic tools (the wrench) → Donation campaign**. This opens the management landing page for that topic; the topic is already fixed, with nothing to type and no ID to look up.
3. The landing resolves the current state: with no campaign yet (and permission to manage the shell) you get the create form; with a campaign already there you get its summary plus only the actions you are authorised for.
4. On the create/edit form, enter a title, an optional BBCode description, and the target; optionally add a donation link (`http://` or `https://` only); and choose whether to show donor names and the donation count.
5. Save, then **Back to topic** — the box is there.

Campaign actions (create, edit, enable/disable, delete an empty campaign) require `a_donationcampaigns` or `m_donationcampaigns_manage` in that forum. The ACP shows a read-only oversight list of campaigns and admin-only maintenance; it cannot create or edit one.

One campaign per topic, enforced by a unique database index. A campaign cannot be attached to the stub left behind by a moved topic — phpBB treats such a stub as non-existent, so a campaign there would be unreachable.

Donation workflow

1. **Wait for the money to actually arrive.**
2. On the topic, **Topic tools → Donation campaign** to open the management landing, then **Add confirmed donation**. The donation ledger — the button and the per-row Edit/Delete controls — is shown only to holders of `a_donationcampaigns` or `m_donationcampaigns_donations` in that forum; a manage-only holder does not see it.
3. Enter the amount received, the date it arrived, and the donor's display name.
4. Decide whether the donor may be named publicly — see [PRIVACY.md](#) before ticking that box.
5. Save. The campaign total is recalculated immediately from all its donations.

Amounts accept both `50.00` and `50,00`. Leave the name empty to record the donation as *Anonymous*. A donation marked private still counts towards the total and the count; only the name is withheld.

The total is always recomputed as the sum of the donation rows, never adjusted by arithmetic, so a figure that has somehow drifted is corrected by the next edit — or immediately, using **Recalculate total** in the ACP.

The ACP shows a read-only oversight list of donations; recording, editing and deleting them happens on the topic.

Known limitations

- No payment processing, and no provider-specific integration. Any provider can be linked to; none is integrated with. This is a design decision, not a gap — see [DEVELOPERS.md](#).
- No public donation form; every entry is made from the topic by an administrator or an authorised forum moderator.
- **Campaigns can only be managed from their topic**. A style that does not provide the `viewtopic_topic_tools_after` template event therefore offers no way to create one — see the prosilver note above.
- **A campaign cannot be moved to another topic** once created.

- Donation dates are date-only and stored as UTC midnight; the board's timezone is not applied to them.
- prosilver only.
- No bulk deletion of donations.
- The ACP campaign list and donation list paginate at 25 rows; there is no search.
- **Not yet reviewed in a browser across styles and screen sizes.**
- Databases other than SQLite and MariaDB have not been executed.
- The phpBB validation and development policies have not been reviewed, so no claim of compliance with them is made.

Licence

GPL-2.0-only. See [license.txt](#).

phpBB is a trademark of phpBB Limited. This extension is not affiliated with or endorsed by phpBB Limited.

Administrator guide

How to run a donation campaign with this extension, and what to be careful about. See the [README](#) for what the extension is, and [PRIVACY.md](#) for what publishing a donor's name means.

Before you start

This extension is a **ledger of payments you have already received**. It does not take money, does not talk to PayPal or a bank, and cannot tell whether a payment arrived. You check your account, and you record what you find.

That means one habit matters more than any setting here:

Record a donation only after the money has actually arrived and you have verified it yourself.

Everything the public sees — the total, the progress bar, the donor list — is built from those records. If you enter a promise, the board will show it as money received.

1. Install and enable

Copy the package to `ext/uflagmey/donationcampaigns/`, then **ACP** → **Customise** → **Extensions** → **Donation Campaigns** → **Enable**.

Enabling creates the tables, the settings, the three permissions and their permission category, and the ACP menu. Disabling later hides everything but keeps your data.

2. Grant the permissions

The extension ships three permissions, grouped under a dedicated **Donation Campaigns** category in the permissions UI.

`a_donationcampaigns` — administrator, granted globally. Full and Standard Administrator roles receive it automatically at installation. It governs the ACP (settings, the read-only oversight lists, and the admin-only maintenance: recalculate a total, hard-delete a non-empty campaign) and, as an override, every frontend campaign and donation action on every forum. An administrator can do everything from the topic as well as from the ACP.

The other two are **moderator, forum-scoped** — you grant them to a user or group on the specific forums whose campaigns they should manage:

- `m_donationcampaigns_manage` lets the holder manage the campaign *shell* on topics in that forum: create, edit, enable/disable, and delete an *empty* campaign. It does **not** grant donation management.
- `m_donationcampaigns_donations` lets the holder manage the donation *ledger* on topics in that forum: add, edit and delete confirmed donations.

The two moderator permissions are **independent**: holding one does not grant the other. `a_donationcampaigns` overrides both.

⚠ `m_donationcampaigns_donations` **exposes personal data.** It reveals donor names, private donor identities and confirmed amounts. Grant it only to people you trust with that information.

⚠ **Granting either moderator permission makes the grantee a forum moderator.** phpBB treats anyone holding an `m_` permission on a forum as a moderator of it: the user or group you grant a donation permission to will appear in that forum's **Moderator** list on topic and forum views and gain moderator standing there. This is how phpBB defines moderators — it is not a silent, hidden grant. Treat granting these permissions as promoting the grantee to a (limited) moderator of that forum.

3. Configure the currency

ACP → Extensions → Donation campaigns → Settings

- **Currency code** — three letters, e.g. `EUR`
- **Currency symbol** — shown next to every amount, e.g. `€`
- **Decimal places** — `2` for most currencies, `0` for yen, `3` for dinar
- **Donors listed** — how many names the public box shows before summarising

⚠ Changing the decimal places later

This is the one setting that can silently misrepresent your figures.

Amounts are stored as whole numbers of the smallest unit — 250.00 € is stored as `25000` cents. The decimal-places setting decides where the separator is drawn when that number is displayed. **It converts nothing.**

Stored	At 2 places	At 3 places
<code>25000</code>	250.00	25.000
<code>870</code>	8.70	0.870

Change it on a board that already has donations and every historic amount is re-read, not re-scaled. The extension will not let you do this by accident: if any campaign or donation exists, it shows a warning and refuses the change until you tick a confirmation box.

Set this correctly **before** recording your first donation. If you must change it afterwards, expect to correct every stored amount by hand.

3a. ⚠ Before you merge or split a topic

Merging a topic into another, or splitting every post out of a topic, deletes that topic — and with it its campaign and every donation record you have entered. phpBB removes the emptied topic automatically, and this extension cleans up alongside it.

There is no warning and no undo, and a moderator can trigger it without holding the donation-campaign permission. If a topic carries a campaign, move posts *into* it rather than merging it away, or export the donation list first.

This is a known defect scheduled to be fixed before the final 1.0 release.

4. Create a campaign

Campaigns are created from the topic they belong to. There is no way to create one from the ACP, and no field anywhere that asks for a topic number.

Open the topic, then:

Topic tools (the wrench) → Donation campaign

That opens the campaign form with the topic already fixed. The same menu entry appears on every topic and always reads *Donation campaign*: if the topic has no campaign you get a new one to fill in, and if it already has one — even a disabled one — you get that campaign to edit. Which of the two you see is decided when you click, not when the page was drawn, so it is never out of date.

The entry shows to anyone who can manage the campaign shell **or** the donations in that forum — administrators (via the `a_donationcampaigns` override) and forum moderators holding either `m_donationcampaigns_manage` or `m_donationcampaigns_donations` for that forum. It is invisible to everyone else, including guests.

Field	Notes
Title	The heading of the public box
Topic	Shown, not editable. A campaign belongs to its topic for life
Description	Optional. BBCode, smilies and links are permitted
Target amount	250.00 or 250,00 . Must be above zero
Donation link	Optional. Must be a full <code>http://</code> or <code>https://</code> address
Link text	The words on the public button, for example <i>How to donate</i> or <i>Über PayPal spenden</i> . Required whenever a donation link is set
Enabled	Untick to hide the box while keeping the data
Show donor names	See PRIVACY.md first
Show donation count	Shows how many donations, without naming anyone

Save, and the page offers **Back to topic**, where the box now appears.

If someone else created or deleted a campaign for that topic while you had the form open, nothing you typed is saved. The form comes back with the current state and says what happened, so a colleague's campaign is never overwritten by a form that was drawn before theirs existed.

About the donation link

The link is rendered as a public button. Only `http://` and `https://` are accepted — `javascript:`, `data:` and protocol-relative addresses are rejected, and are also re-checked before rendering.

The extension does not visit the address or check that it works. Point it somewhere you control and have tested.

The button's wording is yours to choose per campaign, so it can describe what actually happens next — *Request bank details* when the link opens a page of instructions, *Über PayPal spenden* when it opens a payment page. The button is a link and nothing else: the extension embeds no provider form, loads no provider script, shows no logo

and accepts no HTML. Clicking it takes the reader off the board, and you still confirm the money by hand and record it as a donation afterwards.

A link with no text would be a button with nothing written on it, so the extension refuses to save that combination rather than inventing a label.

⚠ About publishing payment details

Nothing here stops you putting your bank details in the campaign description, and people do. Understand what that means before you do it: the description is shown on a public topic that guests and search engines can read, and account identifiers published that way are routinely harvested and used in fraudulent payment requests.

If your board needs to publish payment details, treat it as an operational and privacy decision in its own right — not as a side effect of filling in a form. A donation link to a page you control is usually the safer shape.

5. Record a payment

Only after it has arrived and you have checked.

Recording happens **from the topic**, not the ACP. Open the topic, then **Topic tools** → **Donation campaign** to reach the management landing, and click **Add confirmed donation**. The donation ledger is shown only to an administrator or a holder of `m_donationcampaigns_donations` for that forum; a manage-only moderator does not see it.

Field	Notes
Amount received	The amount that actually arrived. <code>50.00</code> or <code>50,00</code>
Payment received on	The date the money arrived , not today's date
Donor	The name to show publicly. Leave empty for <i>Anonymous</i>
Show donor publicly	Untick to count the donation without naming the donor

Saving recalculates the campaign total immediately.

6. Decide whether to name the donor

Three settings interact:

1. **Campaign** → **Show donor names** — the master switch for that campaign.
2. **Donation** → **Show donor publicly** — per donation.
3. **An empty donor name** — always displayed as *Anonymous*.

A donation with the public flag off still counts towards the total and the donation count. Only the name is withheld.

Ask the donor before publishing their name. The extension cannot know whether you did. See [PRIVACY.md](#).

7. Edit or delete a confirmed entry

Donations are edited and deleted **from the topic**, from the same management landing, by an administrator or a holder of `m_donationcampaigns_donations` for that forum. Editing recalculates the total. So does deleting — the

confirmation names the amount and donor so you can see what you are about to destroy.

Deleting a donation is permanent and there is no undo. A donation cannot be moved to another campaign; delete it and record it under the right one.

Deleting a campaign — the policy differs by where you do it:

- **From the topic**, only an *empty* campaign (no confirmed donations) can be deleted, by an administrator or a holder of `m_donationcampaigns_manage`. A non-empty campaign is refused there — disable it instead, or ask an administrator to delete it.
- **From the ACP**, an administrator (`a_donationcampaigns`) *may* hard-delete a *non-empty* campaign, cascading all of its donations. This is the deliberate difference: the ACP is the admin's tool for that case, and the confirmation names the campaign and its donation count.

Either way, deletion is permanent with no undo, and the total is always recomputed from the donation rows.

8. Recalculate a total

Campaigns → Recalculate total

The displayed total is a cached copy of the sum of that campaign's donations. It is recomputed on every add, edit and delete, so it should never drift. Use this action if you have edited the database directly, restored a partial backup, or simply want to confirm the figure — it recomputes from the donation rows and tells you the value before and after.

It is safe to run at any time and never changes a donation.

9. Disable or archive a campaign

Untick **Enabled** and save. The box disappears from the topic; every donation record is kept, and the campaign is still there on the topic, where you can edit it, add donations to it, or re-enable it from the management landing. The ACP lists it read-only.

Use this when a campaign has finished but you want the records.

9a. Where actions are logged

"Who did what, and where" is read from the log the entry lands in, which depends on the surface the action was performed on.

- **Actions taken from the topic** — creating, editing, enabling/disabling or deleting a campaign, and adding, editing or deleting a donation — are recorded in the **moderator log**, scoped to that forum and topic. Read it in the MCP (**Moderator Control Panel** → **Forum logs**).
- **Actions taken in the ACP** — hard-deleting a campaign, recalculating a total, and changing settings — are recorded in the **administrator log** (**ACP** → **Maintenance** → **Logs**).

10. What happens when a topic or forum is deleted

This is the part worth knowing before someone tidies up the board.

Action	Effect on the campaign
Topic soft-deleted (moderator's "delete")	Nothing. The topic is recoverable, so the campaign and its donations are kept
Topic permanently deleted	The campaign and all its donations are permanently destroyed
Forum deleted with its content	Every campaign in it, and all their donations, are destroyed
Sub-forum deleted	Its campaigns are destroyed; the parent's are not
Topic pruned (manual, moderator, or scheduled)	Same as permanent deletion
All posts in a topic deleted	phpBB removes the emptied topic, so the campaign goes too
User deleted with their posts	Their topics are removed, and campaigns on those topics go with them
Topic moved	Nothing. The campaign follows the topic

Deletion is atomic: either the campaign and all its donations go, or nothing does. No orphaned donation rows are left behind.

There is no warning and no undo. phpBB does not know it is about to destroy a financial record. If a campaign's history matters, export it before deleting its topic — and prefer *disabling* the campaign to deleting the topic.

Purging the extension destroys everything it stores, in every campaign.

Troubleshooting

The box does not appear on the topic. Check the campaign is Enabled, that the campaign is enabled, and purge the cache. The box renders on `viewtopic` ; if the topic itself is unreachable, so is the box.

The total looks wrong. Use **Recalculate total**. If it changes, something wrote to the database outside the extension.

Amounts are out by a factor of ten or a hundred. The decimal-places setting was changed after the data was recorded. See step 3.

A donor's name appears when it should not. Check both switches: the campaign's *Show donor names* and that donation's *Show donor publicly*.

Menu entries have vanished. The extension is disabled. Data is intact.

Privacy and data handling

What this extension stores, what it never stores, and what publishing a donor's name actually does.

This document describes **technical behaviour**. It is not legal advice and makes no claim of compliance with GDPR or any other regime. Whether your use of this extension is lawful is a question for you and, if the answer matters, your own advisers.

See also: [administrator guide](#) · [README](#)

What is stored

Two tables, and nothing else.

Campaigns — `phpbb_ufdc_campaigns`

Column	What it holds
<code>campaign_id</code>	Internal identifier
<code>topic_id</code>	The phpBB topic the campaign is attached to
<code>campaign_title</code>	Title, as the administrator typed it
<code>campaign_desc</code>	Description, plus three columns of phpBB BBCode metadata
<code>target_amount</code>	Target, as a whole number of the smallest currency unit
<code>collected_amount</code>	Cached sum of the campaign's donations
<code>campaign_enabled</code>	Whether the public box is shown
<code>show_donor_names</code>	Whether donor names may be listed
<code>show_donation_count</code>	Whether the number of donations is shown
<code>external_url</code>	Optional donation link
<code>campaign_created</code> , <code>campaign_updated</code>	Timestamps

Donations — `phpbb_ufdc_donations`

Column	What it holds
<code>donation_id</code>	Internal identifier
<code>campaign_id</code>	The campaign it belongs to
<code>donation_amount</code>	Amount, as a whole number of the smallest currency unit
<code>donor_name</code>	A free-text display name, typed by the administrator
<code>donation_time</code>	The date the payment was received

Column	What it holds
<code>donation_public</code>	Whether the name may be shown
<code>donation_created</code> , <code>donation_updated</code>	Timestamps

Four board settings are also stored: currency code, symbol, decimal places, and how many donors the public box lists.

What is not stored

None of the following exists anywhere in this extension — not in a column, not in a log, not in a cache:

- Bank account numbers, IBANs, BICs, sort codes
- Payment credentials of any kind
- PayPal, Stripe or any other transaction or reference identifier
- Payment-provider callbacks or webhook payloads
- Card numbers or any card data
- Proof-of-payment files, screenshots or attachments
- Donor email addresses, postal addresses or telephone numbers
- Donor IP addresses
- Any link between a donation and a phpBB user account
- Private internal notes — there is no notes field

A donation record is an **amount, a date, and a name someone typed**. That is the whole of it.

The extension makes **no outbound network requests**. It never contacts a payment provider, an analytics service, or any third party. Nothing is sent anywhere.

⚠ These guarantees describe the extension's own fields. An administrator can type anything into the campaign description or a donor name, including data that does not belong on a public page. The extension cannot prevent that — see below.

The donor name is the sensitive part

Everything else here is bookkeeping. The donor name is the one field that can identify a living person, and the extension makes publishing it a single checkbox. That deserves stating plainly:

The extension can publish a real person's name on a page that guests can read and search engines can index. It cannot know whether that person agreed. You hold that responsibility, not the software.

The name is free text: it can be a first name, an initial, a pseudonym, a company, or a full legal name. What appears is exactly what was typed.

What "public" means

A campaign box appears on a normal topic. If guests can read that forum, then the donor list is visible to anyone on the internet and can be crawled, cached and archived by search engines. Removing a name later does not remove it from caches and archives that already hold it.

The three controls

1. **Leave the donor name empty.** The donation is recorded and counted, and the box shows *Anonymous*. Nothing identifying is stored at all.
2. **Untick "Show donor publicly"** on the donation. The name is stored but never rendered publicly. The donation still counts towards the total and the donation count.
3. **Untick "Show donor names"** on the campaign. No names are listed for that campaign, whatever the individual donations say.

A private donation is not hidden from the *figures* — its amount is included in the total and in the count. Only the name is withheld. If a donor must not be counted at all, do not record the donation.

Before you publish a name

Ask the donor. The safe default is to record donations anonymously and add a name only when someone has actually asked to be named.

Deletion and erasure

Request	What to do
"Remove my name, keep the donation"	Edit the donation, clear the donor name. It becomes <i>Anonymous</i> ; the total is unchanged
"Do not show my name publicly"	Edit the donation, untick <i>Show donor publicly</i> . The name is retained but never rendered
"Delete my donation entirely"	Delete the donation. The campaign total is recalculated immediately

Clearing the name is usually what people want: the financial record stays accurate and nothing identifying remains.

Deletion is immediate and permanent. There is no archive, no soft-delete and no undo for donation records.

Cascade behaviour

Campaign and donation data is destroyed automatically when the topic it is attached to disappears:

Event	Effect
Topic soft-deleted	Data kept — the topic is recoverable
Topic permanently deleted , or pruned	Campaign and all its donations destroyed
Forum deleted with its content	Every campaign in it destroyed, with all donations
Sub-forum deleted	That sub-forum's campaigns destroyed
All posts in a topic deleted	phpBB removes the topic, so the campaign goes too
User deleted with their posts	Their topics go, and campaigns on those topics with them

Deletion is atomic — donations are removed before their campaign, so no donation row is ever left pointing at a campaign that no longer exists.

Note that deleting a phpBB **user** does not by itself remove donations naming that person: the donor name is free text with no link to an account. Handle those separately.

Uninstalling

- **Disable** — data is kept and nothing is shown. Reversible.
- **Purge** — every campaign, every donation, the settings and the permission are destroyed. Not reversible.

Back up before purging.

Logging

Recording, editing and deleting a campaign or donation writes an entry to phpBB's **admin log**, which administrators can read. Those entries include the campaign title, or the donation amount and donor name.

That means a donor name can survive in the admin log after the donation itself is deleted. If an erasure request has to reach the log too, clear it through **ACP** → **Maintenance** → **Admin log**; the extension does not manage that log's retention.

No logging happens on the public side. Viewing a topic with a campaign records nothing.

Donation Campaigns — Release Notes

1.0.0-beta1

Status: First public beta of the new, frontend-based management architecture — campaign and donation management happen on the topic, under forum-scoped moderator permissions, rather than in the ACP. See *Known limitations* and the release-blocking item below before deploying to a production board.

Requires phpBB **3.3.16 or later** (below 4.0). prosilver only. PHP **8.2 or newer**.

What the extension does

An administrator, or an authorised forum moderator, attaches a fundraising campaign to a topic. The topic then shows a box above its first post with a title, an optional description, a target amount, the amount collected, a progress bar, and — optionally — the donation count and the names of donors who agreed to be listed.

The extension **records confirmed donations; it does not process payments**. The public total is the sum of donation records entered by hand after money actually arrived. An optional button links to wherever a board chooses to take payment (PayPal, a bank-details page, another topic); the extension integrates with no provider.

Headline change — management moved from the ACP to the topic

Campaigns *and* donations are now managed from the topic, not the ACP. The topic-tools **Donation campaign** entry opens a management landing (`/app.php/donationcampaigns/topic/{topic_id}`) that resolves state on arrival: the create form when there is no campaign yet, otherwise a campaign summary plus only the actions the viewer is authorised for. The service layer and the `Repository → Service` core are unchanged; the ACP module's write paths were replaced by thin frontend controllers. See ADR-015 in [DEVELOPERS.md](#).

Three permissions

Grouped under a dedicated **Donation Campaigns** permission category:

- `a_donationcampaigns` — administrator, global. Full ACP access, the admin-only maintenance, and an override for every frontend action on every forum. Granted to the admin roles on install.
- `m_donationcampaigns_manage` — moderator, forum-scoped. Manage the campaign *shell* on topics in that forum: create, edit, enable/disable, delete an *empty* campaign. Does not grant donation management.
- `m_donationcampaigns_donations` — moderator, forum-scoped. Manage the donation *ledger*: add, edit, delete confirmed donations.

The two `m_` permissions are independent; `a_donationcampaigns` overrides both.

 `m_donationcampaigns_donations` **exposes personal data** — donor names, private donor identities and confirmed amounts. Grant it only to people trusted with that information.

 **Granting either `m_` permission makes the grantee a forum moderator.** phpBB treats anyone holding an `m_` permission on a forum as a moderator of it, so the grantee appears in that forum's Moderator list and gains

moderator standing there. This is inherent to phpBB and is a real (if limited) promotion, not a hidden grant.

Deletion policy — split by surface

- **From the topic**, only an *empty* campaign can be deleted; a non-empty one is refused (disable it, or ask an administrator).
- **From the ACP**, an administrator *may* hard-delete a *non-empty* campaign, cascading its donations, behind a confirmation that names the campaign and its donation count.

Donations are deleted individually from the topic behind a confirmation naming the amount and donor. All deletion is permanent; totals are always recomputed from the donation rows.

Audit by surface

Frontend actions (campaign create/edit/enable/disable/delete; donation add/edit/delete) are written to the **moderator log**, scoped to the forum and topic, and read in the MCP. ACP actions (hard-delete, recalculate, settings) are written to the **administrator log**. The log an entry lands in tells you where the action was performed.

The ACP is now read-only oversight plus admin maintenance

ACP → Donation campaigns keeps Settings, a read-only Campaigns list and a read-only Donations list. Its former create/edit forms are gone; the list rows link out to the frontend routes on the topic. It retains admin-only maintenance: **Recalculate total** and campaign hard-delete, both requiring `a_donationcampaigns`.

Localisation

The extension ships **English and German** language packs (`language/en` and `language/de`), kept at parity by a test.

Major architectural decisions

Recorded in full in [DEVELOPERS.md](#); summarized here.

ADR	Decision
001	Target phpBB 3.3.x only
002	Money as integer minor units; one board-wide currency
003	The collected total is denormalised but always recalculated from <code>SUM()</code> , never delta-adjusted
004	Donors are free text, not references to forum users
005	Two services and two repositories; business logic in services, persistence in repositories
006	Public box renders at the <code>viewtopic_body_poll_before</code> template event
007	Two deletion listeners feed one unified cleanup operation
008	A single <code>a_donationcampaigns</code> permission (superseded by ADR-015)

ADR	Decision
009	The concurrent-write race is accepted and mitigated operationally (unique index is authoritative)
010	No JavaScript is required for any essential function
011	Policy and coding-standard violations fail the commit that introduces them
012	Every shared-namespaces identifier carries the package machine name
013	Version 1.0 officially supports prosilver only
014	Campaigns are created only from their topic; the ACP resolves state at request time (superseded by ADR-015)
015	Frontend-primary management with forum-scoped moderator permissions; ACP becomes read-only oversight plus admin maintenance

The layering (`Repository` → `Service` → `Controller` / `Listener` / `ACP Module`) is unchanged. The extension never modifies phpBB core and adds no public write route.

Known limitations

These are deliberate boundaries of the scope, not defects.

- **prosilver only.** A style that does not provide the `viewtopic_topic_tools_after` template event offers no way to manage a campaign, because that is the only management entry point (ADR-013 / 015).
- **No payment processing** and no provider integration, by design. The button links out; nothing is charged, and no transaction data is stored.
- **No public donation form.** Every entry is made from the topic by an administrator or an authorised forum moderator.
- **A campaign cannot be moved to another topic** once created.
- **One campaign per topic**, enforced by a unique index.
- **Donation dates are date-only**, stored as UTC midnight; the board timezone is not applied to them.
- **No bulk deletion of donations.**
- **Campaigns can attach to a soft-deleted or unapproved topic.** The box simply does not render until the topic is visible.

Known issue — RELEASE-BLOCKING before 1.0 final

Merging topics, or splitting every post out of a topic, deletes that topic — and with it its campaign and every confirmed donation record.

phpBB empties the source topic during a merge or full split, then removes it via `delete_topics()` (`functions_admin.php:2058`), which fires `core.delete_topics_before_query` — the event this extension's cascade listens to. The cleanup then removes the campaign and its donation rows.

The deletion is **silent** and **irreversible**, and it can be triggered by a moderator who does not hold any donation permission. These are records of money actually received.

The cascade itself is correct in isolation — an orphaned campaign pointing at a dead topic would be worse. The fix belongs at the level of distinguishing a topic being *destroyed* from a topic being *emptied into another*, and must be designed rather than patched. Documented for administrators in `docs/ADMIN_GUIDE.md` and for developers, with the verified call chain, in [DEVELOPERS.md](#).

Upgrade / migration

A new migration adds the two forum-scoped moderator permissions, `m_donationcampaigns_manage` and `m_donationcampaigns_donations`. **Neither is granted to anyone on install.** Existing `a_donationcampaigns` access is unchanged, so upgrading administrators keep working exactly as before, and **no moderator gains any access until a board owner explicitly grants the opt-in permissions** on the forums they should manage.

Existing campaigns and donations continue to work with no data step.

Why beta

The management architecture is new — a frontend surface, new controllers, and a new permission model — so this release is a beta, to gather real-world verification of that surface before a final 1.0. The reasoning is recorded in full as ADR-015 in [DEVELOPERS.md](#).

Developer notes

Architecture, the contracts that hold it together, and how it is tested.

See also: [README](#) · [administrator guide](#) · [privacy](#)

Package

Composer name	<code>uflagmey/donationcampaigns</code>
Directory	<code>ext/uflagmey/donationcampaigns/</code>
Namespace	<code>uflagmey\donationcampaigns</code>
Licence	GPL-2.0-only

Identifiers that enter a shared phpBB namespace carry the full `donationcampaigns` prefix — config keys, the permission, service ids, language keys, template variables, form keys, log keys, CSS classes.

Physical database identifiers are the one exception, using the abbreviated stem `ufdc`: `phpbb_ufdc_campaigns`, `phpbb_ufdc_donations`. Oracle caps identifiers at 30 bytes, phpBB 3.3 supports Oracle, and phpBB validates column and index names but *not* table names — an over-long table name fails as a raw driver error during installation. `phpbb_donationcampaigns_campaigns` is 33 bytes. A test asserts every physical identifier still fits.

Layers

ACP module / event listeners	coordinate only – no SQL, no rules
↓	
services	business rules, transaction boundaries
↓	
repositories	persistence only – all SQL lives here

- **Repositories** hold every query. They cast values to their intended PHP types at the boundary, so no consumer has to remember the database returns strings. Not-found contract: single reads return `null`, list reads return an empty array, counts and sums return `int 0`.
- **Services** own validation, ordering and transactions. They take the database handle *only* to open transactions and issue no SQL of their own.
- **Listeners and the ACP module** read input, call a service, and render.

A test suite reads the source and fails the build if SQL appears outside a repository, or if the database handle is used for anything but a transaction.

Money

Every amount is an **integer number of the smallest currency unit**. 10.00 EUR is `1000`. No float, no double, no `*` `100`, no `/ 100`, anywhere.

Parsing and formatting are string operations in `currency_formatter`, because the obvious implementation loses money on ordinary input: `(int) ('8.70' * 100)` evaluates to `869`. The test table includes that case, and a 2001-iteration round-trip proves parse/format is lossless.

An architectural test forbids `(float)`, `(double)`, `floatval`, `round()` and `number_format` in production code.

The campaign total

`campaigns.collected_amount` is a **cache**. The donation rows are the truth.

After every donation create, edit and delete, the total is recomputed as `SUM(donation_amount)` and written inside the same transaction. It is never adjusted by arithmetic — no `total + amount`, no `total - amount`, no `total + (new - old)`.

Deltas compound: one missed rollback or one row changed out of band and the figure is permanently wrong with nothing able to detect it. Recomputing is self-healing — any single successful mutation repairs whatever drift preceded it. Four tests seed a deliberately corrupted total and assert that a subsequent add, edit, delete or explicit recalculation fixes it, and a 200-operation randomised sequence asserts, after every committed operation, that every campaign's stored total equals a freshly computed `SUM()`.

`collected_amount` is absent from the services' writable-field lists, so it can never arrive from request input.

Transaction boundaries

Method	Boundary
All reads, and validation	None
<code>campaign_service::create_campaign()</code> / <code>update_campaign()</code>	None — one statement
<code>campaign_service::delete_campaign()</code>	Own
<code>campaign_service::purge_for_topics()</code>	Own; none at all when nothing matches
<code>campaign_service::purge_for_forum()</code>	Delegates to <code>purge_for_topics()</code>
<code>donation_service::*</code> (add, edit, delete, recalculate)	Own, via one private helper

phpBB's `sql_transaction('rollback')` is **not** nest-aware: it unwinds the whole active transaction, not just the extension's level. That is deliberate here — a cleanup failure inside core's deletion aborts core's deletion too, rather than leaving orphans.

Confirmed-donation semantics

A donation row is money **already received and verified by an administrator**. The extension processes no payments, contacts no provider, makes no outbound requests, and exposes no public entry point. Version 1.0 supports manually confirmed donations only.

The action button

`external_url` is a destination and `external_link_text` is the label on the button pointing at it. Both are per campaign, and neither carries provider semantics: there is no provider column, no enum, no transaction id, no callback and no embedded markup. A PayPal link and a link to a forum topic explaining bank transfers are the same thing to this code.

The label is plain text with **no BBCode pipeline** — unlike `campaign_desc`, nothing parses it — stored raw and escaped once at output with `|e`. The column is `VCHAR:100`, deliberately narrower than the 255 used elsewhere in the table, because a button label past that length renders badly at any width;

`campaign_service::MAX_LINK_TEXT_LENGTH` matches it and a test asserts the two stay equal. Length is counted in characters, not bytes.

A URL with an empty label is refused by `campaign_service` rather than substituted at render time. Falling back silently would accept a submission the administrator did not mean to make; with no URL the button is not rendered at all, so an empty label is simply unused and permitted.

Decision: no payment-provider integration

The extension intentionally treats payment destinations as ordinary external HTTPS links. No provider-specific integration exists in version 1.0.

This was decided deliberately, not deferred for lack of time. A per-campaign URL plus a per-campaign button label already expresses every destination a board needs, and it does so without the extension knowing what any of them are. PayPal works today: paste the hosted donation link from a PayPal account into the campaign URL and set the label to say so.

A board-wide provider mode was designed and **rejected**. Two reasons decided it. A global setting that supplied the destination would override a campaign that had deliberately been pointed at a bank-information topic, which breaks boards running more than one kind of destination at once. And generating a provider URL from a merchant identifier means adopting that provider's link format — the obvious PayPal one belongs to a deprecated integration, and the current replacement requires provider-hosted setup and external JavaScript, both outside what this extension will load.

Deferred future work, explicitly not planned for 1.0: if a provider ever genuinely needs automation, phpBB's `service_collection` tagging is the mechanism (core selects authentication providers that way in `provider_collection::get_provider()`). Nothing in the current schema blocks it, because campaigns store a destination and a label and never a provider. Until such a provider exists, no registry, interface or abstraction should be added — one implementation behind an interface is not extensibility, it is indirection.

Deletion cascades

Two listeners, because phpBB destroys topics two different ways.

Event	Covers
<code>core.delete_topics_before_query</code>	Topic deletion, ACP/MCP/scheduled pruning, deleting the last post of a topic, deleting a user with their posts
<code>core.delete_forum_content_before_query</code>	Forum and sub-forum deletion

`prune()`, `auto_prune()`, `delete_posts()` (when it empties a topic) and `user_delete()` all funnel into `delete_topics()`, so one listener covers them all. **Forum deletion is the exception** — it removes topics with a

direct `DELETE ... WHERE forum_id` and never calls `delete_topics()`. A test asserts that asymmetry so it fails loudly if core ever changes.

⚠ The forum event's `topic_ids` payload is **not** usable. Core builds it from a join against the attachments table, so it lists only topics that *have* an attachment, repeats a topic once per attachment, and omits ordinary topics entirely. The service resolves the real topic list through `topic_repository` instead. The test fixture reproduces that misleading payload deliberately, so an implementation trusting it fails.

Donations are always deleted before campaigns. Donation rows key on `campaign_id`; reverse the order and the ids become unresolvable and the rows are orphaned permanently. Asserted on the statements actually issued, on both SQLite and MariaDB.

There is deliberately **no denormalised** `forum_id` on campaigns — it would make the cascade trivial and then have to be kept correct on every topic move, forever.

⚠ Known defect: merging or fully splitting a topic destroys its campaign

Release-blocking; a fix is required before 1.0 final. Recorded here rather than left to be rediscovered.

Merging topics, or splitting *every* post out of one, empties the source topic. `move_posts()` then calls `sync('topic', ...)`, which collects the now-empty topic into `$delete_topics ( functions_admin.php:1967 )` and calls `delete_topics() ( :2058 )`. That fires `core.delete_topics_before_query ( :833 )` — the event this extension's cascade listens to — and the campaign and **every confirmed donation record attached to it** are deleted.

Three things make this worse than it first sounds: the deletion is silent, it is irreversible, and it can be triggered by a moderator who does not hold `a_donationcampaigns` and gets no indication that anything was lost. These are records of money actually received.

The cascade itself is not wrong — an orphaned campaign pointing at a dead topic would be worse. The fix belongs at the level of *distinguishing* a topic being destroyed from a topic being emptied into another one, and must be designed rather than patched. It must **not** be addressed by reintroducing a context-free creation form so the campaign can be retyped.

Shadow topics

A campaign cannot attach to the stub left behind by a moved topic. `viewtopic.php?t=<shadow>` answers HTTP 404 "*The requested topic does not exist*" — identical to a topic that never existed — so a campaign there would render nowhere. `topic_repository::topic_exists()` excludes `topic_moved_id != 0`, and validation reports it exactly as a missing topic.

Cleanup deliberately still finds shadows, so a campaign attached to one before this rule is still purged with its forum rather than becoming both unreachable and undeletable.

phpBB integration points

Verified against core source, not inferred from names.

Point	Location	Note
<code>core.viewtopic_assign_template_vars_before</code>	<code>viewtopic.php:775</code>	<code>topic_id</code> in scope, after access checks

Point	Location	Note
<code>viewtopic_body_poll_before</code> (template)	<code>viewtopic_body.html:77</code>	Despite the name, sits outside the <code>S_HAS_POLL</code> conditional that opens two lines later, so it renders on every topic
<code>core.delete_topics_before_query</code>	<code>functions_admin.php:833</code>	Inside core's transaction, opened at 817
<code>core.delete_forum_content_before_query</code>	<code>acp_forums.php:2011</code>	Before core's <code>DELETE ... WHERE forum_id</code> at 2013
<code>viewtopic_topic_tools_after</code> (template)	<code>viewtopic_topic_tools.html:46</code>	Last item in the wrench dropdown. That file is INCLUDE d twice (<code>viewtopic_body.html:44</code> and <code>:418</code>), so anything in it renders in both action bars — the entry therefore carries no id attribute
<code>S_DISPLAY_TOPIC_TOOLS</code>	<code>viewtopic_topic_tools.html:1</code>	Referenced by the wrapper condition and assigned nowhere in core PHP . It exists so an extension can force the dropdown open when no core tool would have
<code>core.permissions</code>	—	Declares the three permissions (<code>a_donationcampaigns</code> plus the two forum-scoped <code>m_donationcampaigns_*</code>) and their category to the ACP UI
<code>overall_header_head_append</code> (template)	—	<code>INCLUDECSS</code> for the stylesheet

The escaping contract

phpBB 3.3 runs Twig with **autoescape disabled** (`phpbb/template/twig/environment.php:79`), so nothing is escaped for you.

Domain scalars are stored raw — campaign titles, topic titles, donor names, external URLs, currency code and symbol. They are read with `$request->raw_variable()`, never `variable()`, which would escape on input and store `&` for an administrator who typed `&`.

Escaping happens at output, in exactly two places:

1. **Templates.** Every administrator-controlled scalar carries Twig's `|e`. phpBB's lexer accepts a filter suffix on `{VAR}`, so `{VAR|e}` compiles to `{{ VAR|e }}` — the idiom core uses in `prosilver/attachment.html`. No `|raw`, ever, and global autoescape is never enabled.
2. **Sinks with no template.** `confirm_box()` renders `{MESSAGE_TEXT}` raw, and the ACP log viewer `sprintf`s log parameters into a language string and prints the result unescaped (`phpbb/log/log.php:665–710`, rendered by `{log.ACTION}`). Both are escaped immediately before the message is built, with phpBB's `utf8_htmlespecialchars()`. The escaped value is never written back to the database.

`U_*` URL variables are assigned already attribute-safe and must **not** carry `|e`, or they would be escaped twice.

A test forbids any direct `htmlspecialchars()` call in production code and requires `|e` on every administrator-controlled scalar in every template.

The BBCode description

The campaign description is the one formatted field and takes the **opposite** contract:

input	<code>\$request->variable(..., true)</code>	escapes
	<code>description_formatter::for_storage()</code>	phpBB's <code>generate_text_for_storage()</code>
storage	<code>text + uid + bitfield + flags</code>	stored together; meaningless apart
display	<code>for_storage's result → for_display()</code>	rendered with NO <code> e</code>
edit	<code>for_edit()</code>	decoded back to source for the textarea

`generate_text_for_display()` does **not** sanitise — it censors and parses BBCode and assumes the text was escaped on the way in. Escaping its output again would render an administrator's formatting as visible tags.

Encoding lives in `campaign_service`, not the ACP module, so no caller can store a raw description. The BBCode metadata columns are absent from the writable fields: they are produced by the encoder, never accepted from a request, because a uid describing markup the text does not contain is how a payload gets past the display path.

`description_formatter` is a thin seam over phpBB's three content functions, which resolve `text_formatter.parser` from the container and therefore need a booted board. Unit tests substitute a fake and assert the service's *policy*; the real pipeline is exercised on the Docker board.

phpBB 3.2+ stores BBCode as **s9e XML** (`<r>...<B><s>[b]</s>...</B></r>`) and leaves `desc_bbcode_uid` and `desc_bbcode_bitfield` empty. That is correct, not a bug; the columns are still required by the display and edit functions.

ADR-014 — Campaigns are created only from their topic, and the ACP resolves state at request time

Superseded by ADR-015. Management no longer lives in the ACP: campaign and donation actions moved to frontend controllers reached from the topic, under forum-scoped moderator permissions. ADR-014 is kept as the historical record of the topic-context creation decision, whose reasoning ADR-015 carries forward.

Creation used to mean reading a topic's numeric id out of its address and typing it into the ACP. Nothing else in phpBB asks that, and the field it was typed into was the one place a campaign could be pointed at the wrong topic.

The entry point is now a single neutral **Donation campaign** item in the topic tools menu. It carries no verb, and the URL carries no action — only the topic. The ACP resolves what to do when the request arrives:

no campaign for this topic	-> create
a campaign, enabled or disabled	-> edit that campaign

Why neutral rather than "Add" / "Manage". A topic page is rendered once and may be clicked much later. Any verb decided at render time is a claim about the past: a campaign can be created, deleted or disabled in between. Resolving on arrival makes stale pages, disabled campaigns, concurrent creation and hand-edited URLs one mechanism instead of four guards. It also keeps the listener free of a campaign lookup, which matters because it runs on every topic view on the board.

Topic context suppresses the action parameter entirely. With a topic in the URL, `action` is not read at all, so `t=10&action=delete&campaign_id=1` cannot be expressed rather than being caught.

The expected-state guard. Resolving on arrival alone would silently turn a lost create race into an edit, overwriting the campaign that won with values typed for one that did not exist. The form records which campaign it was drawn for; on a mismatch nothing is written and the form is redrawn with an explanation. The recorded id is untrusted but can only ever *downgrade* a write to a re-render — the campaign written is always the resolved one.

Two permissions. `a_` is a real ACL option, not a prefix wildcard (`schema_data.sql:411`), so an administrator holding `a_donationcampaigns` without `a_` is refused by `adm/index.php` with a 403. Both are checked before the link renders, and both checks are visibility only — `main_module::main()` still enforces access itself.

The module identifier is derived, never written. phpBB maps a class name to a URL token by replacing backslashes with dashes (`functions_module.php:1141-1150`). Since this link is the only route to creation, a stale literal would remove the feature rather than degrade it, and would fail only for administrators, only at runtime. Two tests cover it.

Links out of the ACP must carry `$phpbb_root_path`. This code runs inside `adm/`, where a bare `viewtopic.php` resolves to `/adm/viewtopic.php`. Every unit test asserting "the URL contains `t=30`" passes either way; only following the link on a running board shows it. All such links go through one helper and a regression test asserts the root path is present.

Consequence, accepted deliberately. A style that does not provide `viewtopic_topic_tools_after` offers no way to create a campaign at all. Under ADR-013 that style is outside v1.0's supported presentation scope. A weaker ACP form was explicitly rejected as a fallback, because it would have preserved the raw topic-id workflow this decision exists to remove.

ADR-015 — Frontend-primary management with forum-scoped moderator permissions

Campaign **and** donation management moved out of the ACP into frontend controllers reached from the topic. The topic-tools **Donation campaign** entry now opens a management landing (`/app.php/donationcampaigns/topic/{topic_id}`) that resolves state on arrival — the create form when there is no campaign and you may make one, otherwise a campaign summary plus only the actions you are authorised for. This supersedes ADR-014.

Why. A public extension needs *forum moderators*, not only board administrators, to run donations — the person who manages a fundraising topic is usually its moderator, not someone with ACP access. The old model could not express that: it had a single global `a_donationcampaigns` and did every write in the ACP, so the only way to let someone manage donations was to make them an administrator of the whole board. The ACP-only shape also did not feel like native phpBB, where topic-level work happens on the topic.

The three-permission model, forum-scoped. One global admin permission plus two forum-scoped moderator permissions:

- `a_donationcampaigns` — global admin, granted to the admin roles on install. Full ACP access and an override for every frontend action on every forum.
- `m_donationcampaigns_manage` — forum-scoped. The campaign shell: create, edit, enable/disable, delete an *empty* campaign.
- `m_donationcampaigns_donations` — forum-scoped. The donation ledger: add, edit and delete confirmed donations.

The two `m_` permissions are independent and neither is granted on install; `a_donationcampaigns` overrides both. All three sit in a dedicated **Donation Campaigns** permission category. Authorisation is checked per forum, resolved from the campaign's topic.

The layering did not change. `Repository → Service → Controller` mirrors the old `Repository → Service → ACP module`: the service layer was left untouched. The frontend controllers are thin coordinators that read input, call the same services and render, exactly as the ACP module did. No business rule moved into a controller.

Deletion policy is split deliberately. From the topic only an *empty* campaign may be deleted; a non-empty one is refused, to protect records of money received from a forum moderator's cleanup. The ACP keeps an admin-only hard-delete that cascades a *non-empty* campaign's donations — that is the administrator's tool for the case the frontend refuses. Donations are deleted individually behind a confirmation naming the amount and donor.

Audit is by surface. Frontend actions (campaign create/edit/enable/disable/ delete; donation add/edit/delete) are written to the **moderator log**, scoped to the forum and topic, so they surface in the MCP alongside other moderator activity. ACP actions (hard-delete, recalculate, settings) are written to the **administrator log**. The log an entry lands in identifies where the action was performed.

Denial is a uniform 404. An unauthorised or malformed request — a forum the caller cannot manage, a missing topic or campaign, a mismatched id — answers with the same "not found" response rather than distinguishing "forbidden" from "does not exist", so the frontend never discloses whether a given campaign or donation exists to someone not entitled to know.

The ACP became read-only oversight plus admin maintenance. It keeps settings, a read-only campaign list and a read-only donation list, and the admin-only maintenance (recalculate a total, hard-delete a non-empty campaign). Its former create/edit forms are gone; the list rows link out to the frontend routes on the topic.

Consequence, accepted deliberately. phpBB treats any holder of an `m_` permission on a forum as a moderator of it, so granting either donation permission lists the grantee among that forum's moderators and gives them moderator standing there. This is inherent to phpBB's definition of a moderator and is documented for board owners as a real (if limited) promotion, not a hidden grant.

Styles

prosilver only, for version 1.0 (ADR-013). Templates under `styles/prosilver/template/`, stylesheet under `styles/prosilver/theme/`, included with `INCLUDECSS` from a header template event.

No inline CSS, no inline JavaScript, and **nothing requires JavaScript** — the progress bar's width is a CSS class per five-percent step rather than an inline style. All custom classes carry the `donationcampaigns-` prefix. No fixed widths, no absolute positioning.

Tests

```

php vendor/bin/phpunit                                # everything
php vendor/bin/phpunit --testsuite unit               # pure logic + architectural guards
php vendor/bin/phpunit --testsuite migration          # schema, config, permission, modules
php vendor/bin/phpunit --testsuite repository
php vendor/bin/phpunit --testsuite service           # rules, transactions, total integrity
php vendor/bin/phpunit --testsuite event             # listeners + shipped prosilver assets
php vendor/bin/phpunit --testsuite acp               # ACP modules + adm templates
php vendor/bin/phpcs --standard=phpcs.xml
find ext -name '*.php' -print0 | xargs -0 -n1 php -l

```

`phpbb_database_test_case` is unusable — it extends `PHPUnit\DbUnit\TestCase`, and dbunit is abandoned and incompatible with PHPUnit 9. Database tests therefore build their schema from phpBB's own baseline migration (`\phpbb\db\migration\data\v30x\release_3_0_0`) and drive phpBB's real tools. The 3.0.0 baseline predates the 3.1 visibility columns, so fixtures that drive core's deletion paths add them explicitly.

Integration tests call **core's own functions** — `delete_topics()`, `delete_posts()`, `prune()`, `auto_prune()`, `acp_forums::delete_forum()` — through a real dispatcher with the listeners subscribed, rather than invoking the listeners directly. The assumption worth testing is not that our code works but that core reaches it.

Database coverage

Engine	Status
SQLite 3	Every automated test
MariaDB 10.11	Live integration: topic deletion, forum deletion with and without attachments, donations-first ordering observed in the query log, no orphans, migration idempotence — and the <code>mysqli</code> branch of forum deletion that SQLite never executes
MySQL, PostgreSQL, MS SQL, Oracle	Not executed. DBAL-only SQL, so expected to work; not verified

Local integration environment

A disposable Docker board (phpBB 3.3.17, PHP 8.2, MariaDB 10.11) is used to verify UI work against a real installation, since automated tests cannot see a template that fails to compile or a page that renders wrongly.

It lives outside the extension package and is not part of the repository. Its setup notes live with it. **No credentials, hostnames or local paths belong in this repository.**

EPV and coding standards

```
docker compose exec web /opt/epv/vendor/bin/EPV.php run --dir=/var/www/html/phpBB/ext/
```

EPV must be pointed at the directory *containing* `vendor/package/`, not at the extension root, or it reports a spurious packaging error. The current state is **0 fatal, 0 errors, 0 warnings, 0 notices**.

Two of its rules shaped the code:

- `htmlspecialchars()` is an **Error** in EPV's eyes, whatever the context. Hence `|e` in templates and `utf8 htmlspecialchars()` at sinks.
- **Its SQL-injection check is a regex** requiring `' . $`, skipped when the line contains `sql_in_set`, `sql_escape` or several other DBAL helpers. An inline `(int)` cast breaks the pattern — which is why queries are written `= ' . (int) $id` rather than casting on an earlier line.

EPV passing is not proof of submission-policy compliance. It is a mechanical scanner. The phpBB validation and development policies have not been reviewed for this extension, and no claim about them is made.